BE Ma312 : Une courte introduction à la transformée de Fourier discrète

ALAUX Florian
~~LENDROIT Julie~~
TORREGROSSA Laurie
~~LECUYER Thomas~~
~~VASSIL Jolan~~
N'ont pas participé ou n'ont rien communiqué (détail en mail)

M. Couffignal

Date : 23 octobre 2024

1 La transformée de Fourier discrète

1.1 Des indices modulo N (les groupes cycliques)

Soit $f := (f_1, \cdots, f_N)^T \in \mathbb{C}^N$. Nous voulons réaliser l'analyse de Fourier de vecteur.

Si nous le voyons comme les valeurs d'une fonction 2π-périodique aux temps $t := (t_0, t_1, \cdots, t_{N-1})^T$ tel que :

$$t_j := jh \quad \text{avec : } h = \frac{2\pi}{N} \quad \text{et} \quad j \in [\![0, N]\!]$$

Nous échantillons la fonction ainsi, avec ce vecteur, nous pouvons donc connaître certaines valeurs de f :

$$f_j := f(t_j)$$

Pour réaliser l'analyse de Fourier sur un tel vecteur,nous allons le voir comme les valeurs d'une fonction f :

$$f(t) := \sum_{j=0}^{N-1} f_j \chi_{[t_j, t_{j+1}[}(t)$$

f est 2π-périodique continue par morceaux et vérifiant la condition de Dirichlet

$$\forall x \in]t_j; t_{j+1}[f(x) = \frac{1}{2}(f(x_-) + f(x_+)$$

où $f(x_-$ et $f(x_+)$ sont respectivement les limites à gauche et à droite au point x

On notera par abus encore f la fonction réelle 2π-périodique en escalier correspondante :

$$f(t) := \sum_{j=0}^{N-1} f_j \chi_{[t_j, t_{j+1}[}(t)$$

pour tout $t \in [0, 2\pi[$.

Nous supposons les vecteurs $f \in \mathbb{C}^N$ comme périodiques.

Remarque : Nous retrouvons l'appellation de vecteur cyclique.

Nous pouvons alors montrer grace à l'hypothèse que f est 2π-périodique et que nous avons pour tout $k \in \mathbb{Z}$:

$$f_{k+aN} = f(t_{k+aN}) = f((k + aN)h) = f(kh + aNh) = f(kh + a2\pi) = f(kh) = f(t_k) = f_k$$

$$f_k := f_{k \bmod N}$$

Ainsi, les indices du vecteur $f \in \mathbb{C}^N$ auront la propriété « circulaire/cyclique » précédente.

1.2 Le cadre géométrique discret

1. $\mathbb{C}^N$ un $\mathbb{C}$-espace vectoriel :

Pour montrer que $\mathbb{C}^N$ est un $\mathbb{C}$-espace vectoriel, nous devons vérifier que tout vecteur dans $\mathbb{C}^N$ satisfait les deux propriétés suivantes :

- **Addition vectorielle** : Soient $f = (f_0, f_1, \ldots, f_{N-1})$ et $g = (g_0, g_1, \ldots, g_{N-1})$ deux vecteurs de $\mathbb{C}^N$, alors $h_k = f_k + g_k$ pour $k \in [\![0, 1, \ldots, N-1]\!]$.

- **Multiplication scalaire** : Soit $\alpha \in \mathbb{C}$ un scalaire, alors $\alpha f = (\alpha f_0, \alpha f_1, \ldots, \alpha f_{N-1})$.

Ces opérations respectent les règles essentielles d'un espace vectoriel : la commutativité, l'associativité, la distributivité, il existe un 'zéro' pour l'addition et chaque élément a un opposé pour l'annuler.

Si ces points sont vérifiés, alors $\mathbb{C}^N$ est un $\mathbb{C}$-espace vectoriel.

2. Produit scalaire hermitien :

Vérifions maintenant que $\mathbb{C}^N$ admet un produit scalaire hérmitien défini pour f et g deux vecteurs de $\mathbb{C}^N$ par :

$$\langle f, g\rangle = \frac{1}{N}\sum_{k=0}^{N-1} f_k\overline{g_k}$$

Nous devons vérifier qu'il vérifie les propriétés du produit scalaire :

- **Linearité en f :**

Si $f, g, h \in \mathbb{C}^N$ et $\alpha \in \mathbb{C}$, alors :

$$\langle \alpha f + g, h\rangle = \frac{1}{N}\sum_{k=0}^{N-1}(\alpha f_k + g_k)\overline{h_k} = \frac{1}{N}\sum_{k=0}^{N-1}(\alpha f_k\overline{h_k} + g_k\overline{h_k}) = \alpha\langle f, h\rangle + \langle g, h\rangle$$

- **Conjugaison :**

$$\langle g, f\rangle = \frac{1}{N}\sum_{k=0}^{N-1} g_k\overline{f_k} = \overline{\langle f, g\rangle}$$

- **Positivité :**

$$\langle f, f\rangle = \frac{1}{N}\sum_{k=0}^{N-1} f_k\overline{f_k} = \frac{1}{N}\sum_{k=0}^{N-1} |f_k|^2 \geq 0$$

et $\langle f, f\rangle = 0$ si et seulement si f est le vecteur nul.

On note la norme associée au produit scalaire hermitien par :

$$\|f\|_2 = \sqrt{\langle f, f\rangle} = \frac{1}{\sqrt{N}}\sqrt{\sum_{k=0}^{N-1} |f_k|^2}$$

1.3 Vers la transformée de Fourier discète

Nous allons d'abord faire quelques rappels sur les fonctions de l'espace de Dirichlet. Cette fois, on considère f comme une fonction périodique. L'opérateur de la série de Fourier, que l'on note :

$$\widehat{\bullet} : \mathcal{D}_{2\pi}(\mathbb{R}, \mathbb{C}) \to l^2(\mathbb{Z})$$

est défini ainsi :

$$\widehat{f} := \left(c_k(f) := \frac{1}{2\pi}\int_0^{2\pi} f(t)e^{-ikt}\,dt\right)_{k\in\mathbb{Z}}$$

et est injectif. On note la série de Fourier associée à f :

$$S(f)(t) := \sum_{k=-\infty}^{+\infty} c_k(f)e^{ikt}$$

On fixe un entier $N > 0$, que l'on choisit pair. D'un point de vue pratique, il est impossible de considérer des sommes infinies. On va donc travailler avec une version tronquée de la série de Fourier, limitée à l'ordre N :

$$S_N(f)(t) := \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} c_k(f)e^{ikt}.$$

1.4 La transformée de Fourier discrète

Pour rendre la somme plus agréable à manipuler on commence par ré-indicer cette somme afin de la faire commencer à 0 (commme Python pour les array, c'est drole) en posant $j = k + \frac{N}{2}$. Cela nous donne :

$$S_N(f)(t) := \sum_{j=0}^{N-1} c_{j-\frac{N}{2}}(f) e^{i\left(j-\frac{N}{2}\right)t}.$$

Encore une fois, d'un point de vue pratique, les valeurs de t où l'on souhaite évaluer notre série de Fourier doivent être elles aussi discrètes. Nous supposons donné un vecteur $t := (t_0, t_1, \cdots, t_{N-1})^T$ défini tel que :

$$t_j := jh \quad \text{avec} \quad h = \frac{2\pi}{N} \quad \text{et} \quad j \in [\![0, N]\!].$$

Ce vecteur t représente les points où nous allons «échantillonner» notre fonction.
Pour ce type de vecteur «d'entrée», les valeurs de f que l'on peut connaître sont les :

$$f(t_j) := f_j.$$

D'un autre côté, il nous faut introduire l'équivalent discret des coefficients $c_k := \frac{1}{2\pi}\int_0^{2\pi} f(t)e^{-ikt}\,dt$.
Puisque seuls les f_j sont considérés, on remplace f par sa version discrète :

$$f_d(t) := \sum_{j=0}^{N-1} f(t_j)\,\chi_{[t_j,t_{j+1}[}(t)$$

Pour une telle fonction on a si $t \in [t_j, t_{j+1}[$,

$$f_d(t) = f(t_j) = f_j$$

Plus généralement, on notera pour une fonction g : g_d ou $(g)_d$ sa version discrète.

1.4.1 les coefficients $c_k(f)$ discrets

Nous connaissons les coefficients :

$$c_{k,D}(f) := \frac{1}{2\pi}\int_0^{2\pi} \left(t \to f(t)e^{-i\left(k-\frac{N}{2}\right)t}\right)_D dt$$

Nous allons calculer leur version discrète pour $k \in [\![0, N-1]\!]$:

$$\begin{aligned}
c_{k,D}(f) &= \frac{1}{2\pi}\int_0^{2\pi} \left(t \to f(t)e^{-i\left(k-\frac{N}{2}\right)t}\right)_D dt \\
&= \frac{1}{2\pi}\int_0^{2\pi} \sum_{j=0}^{N-1} f(t_j)\, e^{-i\left(k-\frac{N}{2}\right)t_j} \chi_{[t_j,t_{j+1}[}(t)dt \\
&= \frac{1}{2\pi}\sum_{j=0}^{N-1} f(t_j)\, e^{-i\left(k-\frac{N}{2}\right)t_j} \left(\int_{t_j}^{t_{j+1}} dt\right) \\
&= \frac{1}{N}\sum_{j=0}^{N-1} f(t_j)\, e^{-i\left(k-\frac{N}{2}\right)t_j}
\end{aligned}$$

Ainsi, les coefficients discrets de la transformée de Fourier sont donnés par :

$$c_{k,D}(f) = \frac{1}{N}\sum_{j=0}^{N-1} f(t_j)\, e^{-i\left(k-\frac{N}{2}\right)t_j}$$

En introduisant la racine principale d'ordre N :

$$W_N := e^{-ih} = e^{-i\frac{2\pi}{N}}$$

On peut aussi réécrire $c_{k,D}(f)$ sous la forme :

$$c_{k,D}(f) = \frac{1}{N} \sum_{j=0}^{N-1} f\left(t_j\right) W_N^{\left(k-\frac{N}{2}\right)j}$$

Afin de lier la version discrète du théorème de Dirichlet à la version que nous connaissons, on introduit le vecteur $e_N^{k-\frac{N}{2}}$ dont les composantes sont données par :

$$\left(e_N^{k-\frac{N}{2}}\right)_j = \overline{W_N^{\left(k-\frac{N}{2}\right)j}} = \bar{W}_N^{\left(k-\frac{N}{2}\right)j} = e^{i\left(k-\frac{N}{2}\right)t_j}$$

Ainsi, on peut définir la **transformée discrète de Fourier** par la formule :

$$S_N^D(f)(t) := \sum_{k=0}^{N-1} c_{k,D}(f) e^{i\left(k-\frac{N}{2}\right)t}$$

Pour l'instant, c'est-à-dire avant de démontrer un résultat analogue au théorème de Dirichlet, on appellera cette série la **série de Fourier discrète**.

1.5 Vers le théorème de Dirichlet discret

En utilisant les notations précédentes nous souhaitons montrer la forme discrète du théorème de Dirichlet que l'on s'écrit de la manière suivante :

$$f = \sum_{k=0}^{N-1} \left\langle f, e_N^{k-\frac{N}{2}} \right\rangle e_N^{k-\frac{N}{2}}$$

où f est ici un vecteur de $\mathbb{C}^N$ et $e_N^{k-\frac{N}{2}}$ note le vecteur dont les composantes sont données par :

$$\left(e_N^{k-\frac{N}{2}}\right)_j = \overline{W_N^{\left(k-\frac{N}{2}\right)j}} = \overline{W_N}^{\left(k-\frac{N}{2}\right)j}$$

Comme nous avions démontrer que la famille des exponentielles complexes forme une famille base Hilbertienne, nous allons montrer un résultat « analogue» pour la famille des vecteurs de la forme $e_N^{k-\frac{N}{2}}$.

Un petit lemme

Soit deux entiers / et j tels que : $0 \leq I, j \leq N-1$. Alors avec les notations précédentes :

$$\left\langle e_N^{j-\frac{N}{2}}, e_N^{I-\frac{N}{2}} \right\rangle = \delta_{jl}$$

où δ_{jl} est le symbole de Kronecker.

Preuve

Si on suppose que $I = j$ alors :

$$\begin{aligned}\left\langle e_N^{j-\frac{N}{2}}, e_N^{I-\frac{N}{2}} \right\rangle &= \frac{1}{N}\sum_{k=0}^{N-1} e^{-i(j-l)} \\ &= \frac{1}{N}\sum_{k=0}^{N-1} 1 = 1\end{aligned}$$

Mais si $I \neq j$, alors on a :

$$\left\langle e_N^{j-\frac{N}{2}}, e_N^{I-\frac{N}{2}} \right\rangle = \frac{1}{N}\sum_{k=0}^{N-1} e^{-i(j-I)}$$

On peut réécrire $e^{-i(j-l)}$ avec la formule d'Euler :

$$\begin{aligned}e^{-i(j-l)} &= e^{-i(j-l)} \\ &= \cos((j-l)) - i\sin((j-l)) \text{ Ainsi :} \\ &= 1\end{aligned}$$

$$\left\langle e_N^{j-\frac{N}{2}}, e_N^{I-\frac{N}{2}} \right\rangle = \frac{N}{N} = 1$$

Donc :

$$\left\langle e_N^{j-\frac{N}{2}}, e_N^{I-\frac{N}{2}} \right\rangle = \delta_{jl}$$

Théorème

Pour tout t_j avec $j \in [\![0, N-1]\!]$:

$$S_N^D(f)\left(t_j\right) = f\left(t_j\right)$$

i.e. pour $f = \left(f_0, \cdots, f_{N-1}\right)^T \in \mathbb{C}^N$:

$$f = \sum_{k=0}^{N-1} \left\langle f, e_N^{k-\frac{N}{2}} \right\rangle e_N^{k-\frac{N}{2}}$$

Preuve

$$S_N^D(f)(t_j) = \sum_{k=0}^{N-1} c_{k,D}(f) e^{i(k-\frac{N}{2})t_j}$$

on a :

$$c_{k,D}(f) = \frac{1}{N} \sum_{l=0}^{N-1} f(t_l) W_N^{(k-\frac{N}{2})l}.$$

Donc :

$$\begin{aligned} S_N^D(f)(t_j) &= \sum_{k=0}^{N-1} \left(\frac{1}{N} \sum_{l=0}^{N-1} f(t_l) W_N^{(k-\frac{N}{2})l} \right) e^{i(k-\frac{N}{2})t_j} \\ &= \sum_{l=0}^{N-1} f(t_l) \left(\frac{1}{N} \sum_{k=0}^{N-1} W_N^{(k-\frac{N}{2})l} e^{i(k-\frac{N}{2})t_j} \right) \end{aligned}$$

On réécrit la parenthèse :

$$\frac{1}{N} \sum_{k=0}^{N-1} W_N^{(k-\frac{N}{2})l} e^{i(k-\frac{N}{2})t_j} = \frac{1}{N} \sum_{k=0}^{N-1} e^{-i(k-\frac{N}{2})(l-j)h}.$$

En appliquant le lemme on a :

$$\frac{1}{N} \sum_{k=0}^{N-1} e^{-ik(l-j)h} = \delta_{lj}.$$

Ainsi :

$$S_N^D(f)(t_j) = f(t_j).$$

1.6 Conclusion

On peut alors poser de manière satisfaisante la définition suivante.

Définition

On appelle **Transformée de Fourier discrète** l'application de $\mathbb{C}^N$ dans $\mathbb{C}^N$:

$$\widehat{\bullet}_D : \quad (f_j := f(t_j))_{j \in [\![0,N-1]\!]} \mapsto (c_{k,D}(f))_{k \in [\![0,N-1]\!]}$$

et on appelle **transformée inverse** l'application de $\mathbb{C}^N$ dans $\mathbb{C}^N$:

$$\widehat{\bullet}_D^{-1} : \quad (c_{k,D}(f))_{k \in [\![0,N-1]\!]} \mapsto (f_j = f(t_j))_{j \in [\![0,N-1]\!]}$$

2 Premier algorithme de calcul

2.1 Un algorithme naïf : la forme matricielle

La transformée de Fourier discrète s'écrit **matriciellement**, en notant :

$$f := (f_0, f_1, \cdots, f_{N-1})^T$$

On a d'après la formule des $c_{k,D}(f)$ pour un $k \in [\![0, N-1]\!]$ fixé :

$$\begin{aligned} c_{k,D}(f) &= \frac{1}{N} \sum_{j=0}^{N-1} f(t_j) W_N^{(k-\frac{N}{2})j} \\ &= \frac{1}{N} \left(w_N^{(k-\frac{N}{2})0}, w_N^{(k-\frac{N}{2})1}, \cdots, w_N^{(k-\frac{N}{2})(N-1)} \right)^T f \end{aligned}$$

Ainsi en notant $c := (c_{0,D}, c_{1,D}, \cdots, c_{N-1,D})^T$ et pour tout $(i,j) \in [\![0, N-1]\!] \times [\![0, N-1]\!]$:

$$A_{kj} := w_N^{(k-\frac{N}{2})j}$$

on obtient l'écriture matricielle :

$$\frac{1}{N} Af = c$$

De même comme d'après le théorème de Dirichlet discret on a :

$$\begin{aligned} & S_N^D(f)(t_j) = f(t_j) \\ \Leftrightarrow & \sum_{k=0}^{N-1} c_{k,D}(f) e^{i(k-\frac{N}{2})t_j} = f(t_j) \\ \Leftrightarrow & \left(W_N^{\frac{N}{2}j}, W_N^{-(1-\frac{N}{2})j}, \cdots, W_N^{-(N-1-\frac{N}{2})j} \right)^T c = f(t_j) \end{aligned}$$

On en déduit que la transformée inverse est donnée par :

$$A_{jk}^{-1} = W_N^{-(k-\frac{N}{2})j}$$

i.e :

$$A^{-1} c = f$$

2.2 Linéarité

Propriété

La transformée de Fourier discrète et la transformée de Fourier inverse discrète sont linéaires.

La preuve est immédiate par la linéarité du produit matriciel.

2.3 Une premier algorithme simple

La formule matricielle précédente nous permet d'avoir un premier algorithme de calcul très simple qui consiste à :

— construire la matrice : A où les coefficients sont : $A_{kj} := w_N^{(k-\frac{N}{2})j}$
— puis calculer : $\left(\frac{1}{N}\right) Af$

En guise d'exemple, on donne le code Python suivant. Pour la transformée de Fourier discrète :

```python
def DFT(x):
    N = len(x)
    WN = np.exp(-1j * 2 * np.pi / N)
    vecgen = np.zeros((N, 1)).astype(complex)
    vecgen = (WN ** np.arange(-N/2, N/2)).reshape((N, 1))
    W = np.ones((N, 1))
    for i in range(1, N):
        W = np.block([W, vecgen ** i])
    W = (1/N) * W
    return W @ x
```

et pour la transformée de Fourier discrète inverse :

```
def iDFT(x) :
    N=len(x)
    WNbar = np.exp(1j*2*np.pi/N)
    vecgen = np.zeros((N, 1)).astype(complex)
    vecgen = (WNbar**np.arange(=N/2, N/2)).reshape((N, 1))
    W = np.ones((N, 1))
    for i in range (1, N) :
        W = np.block([W, vecgen**i])
    return W.T @ x
```

Nous verrons des exemples plus loin dans la partie Python

2.4 Un mot sur la reconstruction

On peut également tester la reconstruction du signal :

```
signalrecons = iDFT(TF).real
print("Ecart au vecteur originel :", np.linalg.norm(f(t) = iDFT
(TF)))
```

Pour les entiers N précédents on trouve des erreurs extrêmement faible de l'ordre de 10^{-16}

3 La transformée de Fourier rapide (FFT)

- L'algorithme de transformation de Fourier rapide ou FFT est souvent vu comme l'un des plus importants algorithmes qui existe. On retrouve de nombreuses traces historiques de cet algorithmes, par exemple on le retrouve chez Gauss en 1805, il doit sa notoriété à James W. Cooley et John W. Tukey et leur article « An algorithm for the machine calculation of complex Fourier series » de 1965.
- Nous allons voir une version simple de cet algorithme.
- **Dans la pratique nous utiliserons les versions déjà implémentées dans les librairies.**

3.1 Vers un algorithme rapide

Si N est une puissance de 2 on peut produire un algorithme de calcul extrêmement rapide. Voyons leurs méthodes. On rappelle que W_N est la racine principale d'ordre N :

$$W_N := e^{-ih} = e^{-i\frac{2\pi}{N}}$$

Elle se base sur deux remarques :

- Un calcul simple montre que :

$$\begin{aligned}\left(W_N^{\left(k-\frac{N}{2}\right)}\right)^2 &= W_N^{2\left(k-\frac{N}{2}\right)}\\ &= e^{-i\frac{2\pi}{N}2\left(k-\frac{N}{2}\right)}\\ &= e^{-i\frac{2\pi}{N}2k}e^{i2\pi}\\ &= W_{\frac{N}{2}}^{k}\end{aligned}$$

- Le calcul d'un coefficient discret s'écrit :

$$c_{k,D}(f) = \frac{1}{N}\sum_{j=0}^{N-1} f\left(t_j\right) W_N^{\left(k-\frac{N}{2}\right)j}$$

On pose : $X := W_N^{\left(k-\frac{N}{2}\right)}$ et on note le polynôme :
$p(X) := \frac{1}{N}\sum_{j=0}^{N-1} f_j X^j$. On sépare le calcul des puissances paires et impaires i.e. en posant :

$$\begin{cases} p_{pair}(X) := \dfrac{1}{N}\left[f_0 + f_2 X + ... + f_{N-2} X^{\frac{N}{2}-1}\right] \\ p_{impair}(X) := \dfrac{1}{N}\left[f_1 + f_3 X + ... + f_{N-1} X^{\frac{N}{2}-1}\right] \end{cases}$$

Alors il est immédiat que :

$$p(X) = p_{pair}(X^2) + X p_{impair}(X^2)$$

Ainsi en regroupant les deux remarques on voit que :

— le calcul d'une transformée de Fourier d'ordre N revient à deux calculs de transformée de Fourier d'ordre $N/2$.

— Si N est une puissance de 2, on peut appliquer cette remarque de manière récursive et ainsi obtenir un algorithme extrêmement rapide.

3.2 En utilisant Numpy

Nous allons commencé par tester sous Python avec des sons réels (au format .wav) la validité numérique de ce théorème en utilisant la transformée de Fourier rapide de Numpy (ou Scipy). Attention on rappelle que dans la transformée de Fourier sous Numpy (ou Scipy), la convention a été choisi de mettre le terme 1/N sur la transformée inverse et non sur la transformée. Adapter votre code en fonction de cette convention.

Nous verrons des exemples dans la partie détiée Python.

4 Théorème de Parseval discret et autres propriétés

4.1 Théçrème de Parseval généralisé

Nous allons à présent démontrer la version discète du théorème de Parseval. Soit $A \in \mathscr{M}_N(\mathbb{C})$ une matrice carrée à coefficients complexes. Nous voulons montrer que pour tous vecteurs f et g de $\mathbb{C}^N$, la relation suivante est vérifiée :

$$\langle Af,\ g\rangle = \langle\ f, A^*\ g\rangle$$

Démonstration

- Soit $f = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_N \end{pmatrix}$ et $g = \begin{pmatrix} g_1 \\ g_2 \\ \vdots \\ g_N \end{pmatrix}$ des vecteurs de $\mathbb{C}^N$.

La matrice transconjuguée A^* est définie par $A^* = \overline{A}^T$, où $\overline{A}$ est la matrice obtenue en prenant le conjugué complexe de chaque élément de A.

Le produit Af donne un nouveau vecteur $h = Af = \begin{pmatrix} h_1 \\ h_2 \\ \vdots \\ h_N \end{pmatrix}$ où :

$$h_i = \sum_{j=1}^{N} a_{ij} f_j \quad \text{pour } i = 1, 2, \ldots, N$$

Le produit scalaire $\langle Af,\ g\rangle$ est donné par :

$$\begin{aligned} \langle A\ f,\ g\rangle = \sum_{i=1}^{N} h_i \overline{g_i} &= \sum_{i=1}^{N} \left(\sum_{j=1}^{N} a_{ij} f_j \right) \overline{g_i} \\ &= \sum_{j=1}^{N} f_j \left(\sum_{i=1}^{N} a_{ij} \overline{g_i} \right) \end{aligned}$$

Calculons maintenant A^*g. Les éléments de A^*g sont donnés par :

$$(A^*g)_i = \sum_{j=1}^{N} \overline{a_{ji}} g_j \quad \text{pour } i = 1, 2, \ldots, N$$

$$\langle f, A^*g\rangle = \sum_{i=1}^{N} f_i \overline{(A^*\ g)_i} = \sum_{i=1}^{N} f_i \overline{\left(\sum_{j=1}^{N} \overline{a_{ji}} g_j \right)}$$

En utilisant la propriété des conjugués on peut écrire :

$$\langle f, A^*g\rangle = \sum_{i=1}^{N} f_i \sum_{j=1}^{N} a_{ji} \overline{g_j}$$

Les deux expressions sont bien identiques, nous avons vérifié que :

$$\langle A\ f,\ g\rangle = \langle\ f, A^*\ g\rangle$$

4.2 Produit de convolution discret

Une opération importante que nous n'avons pas encore discrétisée est celle du produit de convolution. On rappelle que la convolution sur $\mathscr{D}_{2\pi}(\mathbb{R}, \mathbb{C})$ est donnée par :

$$f \star g(t) := \frac{1}{2\pi} \int_0^{2\pi} f(t-u) g(u) du$$

pour tout $x \in \mathbb{R}$

Propriété

La propriété de compatibilité à la transformation de Fourier discrète nous dit : pour tout f et g deux vecteurs de $\mathbb{C}^N$ et pour tout $k \in [\![0, N-1]\!]$:

$$c_{k,D}(f \star g) = c_{k,D}(f)c_{k,D}(g)$$

Démonstration

La DFT de $(f \star g)$ est donnée par :

$$\begin{aligned} c_{k,D}(f \star g) &= \frac{1}{N}\sum_{j=0}^{N-1}(f \star g)e^{-i\frac{2\pi}{N}kj} \\ &= \frac{1}{N}\sum_{j=0}^{N-1}\left(\sum_{m=0}^{N-1} f_j - mgm\right)e^{-i\frac{2\pi}{N}kj} \end{aligned}$$

En interchangeant les sommes on a :

$$c_k(f \star g) = \frac{1}{N^2}\sum_{m=0}^{N-1} g_m \sum_{j=0}^{N-1} f_{j-m}e^{-i\frac{2\pi}{N}kj}$$

On pose alors $j' = j - m$:

$$c_k(f \star g) = \frac{1}{N}\sum_{m=0}^{N-1} g_m \frac{1}{N}\sum_{j'=0}^{N-1} f_{j'}e^{-i\frac{2\pi}{N}k(j'+m)}$$

On réécrit de facon à faire apparaitre 2 DFT :

$$c_k(f \star g) = \frac{1}{N^2}\sum_{m=0}^{N-1} g_m e^{-i\frac{2\pi}{N}km} \cdot \sum_{j'=0}^{N-1} f_{j'}e^{-i\frac{2\pi}{N}kj'}$$

Finalement on a bien :

$$c_{k,D}(f \star g) = c_{k,D}(f) \cdot c_{k,D}(g)$$

Remarque

Cette propriété montre que la convolution dans le domaine temporel correspond à une multiplication dans le domaine fréquentiel. C'est une propriété fondamentale de la transformation de Fourier qui est très souvent exploitée comme par exemple en traitement du signal.

4.3 Conséquences pratiques

Le résultat précédent nous donne accès au monde du filtrage numérique :

- En effet filtrer un signal (sonore par exemple) f revient souvent à calculer $f \star h$ où h est le vecteur incarnant le filtre souhaité.
- Or un tel calcul peut être complexe à décrire dans le domaine temporel mais extrêmement simple dans le domaine fréquentiel.

On souhaite créer un filtre coupe-bande i.e. supprimer des fréquences d'un signal. La démarche est la suivante.

- On importe un fichier son au format .wav et si le son est en stéréo (plusieurs colonnes) on sépare ces différentes colonnes afin de travailler séparément sur chacune d'elles.
- On commence par réaliser la transformée de Fourier discrète $\widehat{f_D}$ du son f. On utilisera dans Python la transformée de Fourier rapide réelle np.fft.rfft afin d'accélérer les calculs.

— On multiplie composantes par composantes $\widehat{f_D}$ par un vecteur $\tilde{h}$ ne contenant que des 0 (pour les fréquences à supprimer) et 1 (pour les fréquences à conserver) puis on réalise la transformée de Fourier discrète inverse :

$$\mathscr{F}_D^{-1}\left(\widehat{f_D} \times \tilde{h}\right)$$

On obtient alors un signal filtré. **Attention à bien penser à convertir le résultat final au bon format(i.e. au format du son d'origine : int16, int32 ou float32 avant toute écoute.**

4.4 Remarque sur les sigaux réels

On suppose que $f \in \mathbb{R}^N$ (comme pour les sons que nous étudions). Alors il vient que pour tout $k \in [\![\frac{N}{2}, N]\!]$:

$$c_{k,D}(f) = \overline{c_{-k+N,D}(f)}$$

On suppose que f est un signal temporel et a été échantillonné à la fréquence f_e.

Si f représente un signal audio, alors chaque f_k correspond à une certaine fréquence sonore dans le signal, qui peut être vue comme une note de musique ou un ton.

5 Exemples et Illustrations

Nous allons utiliser l'importation de fichier .wav sous python. Afin de lire et d'entendre des sons nous aurons besoin des librairies Scipy et sounddevice, en particulier le module wavfile. Il faudra donc les importer au début de notre code comme ceci :

```
import sounddevice as sd
import time
from scipy.io import wavfile
```

L'étude suivante est basée sur la variation et l'analyse de signaux. Pour importer un son à étudierdans python, il y a plusieurs possibilités. Par exemple, on peut soit :

- Importer un fichier .wav :

```
# Construit le chemin vers le fichier "drum.wav" dans "Data" que l'on veut ouvrir
file_path = os.path.join('Data', 'drum.wav')
fe, data = wavfile.read(file_path)  # Lis l'audio, récupère la fe et des data audio
sd.play(data, fe)  # joue l'audio récupéré grace aux data et à la fe
```

- Soit enregistrer le son en direct avec le micro de l'ordinateur :

```
fe = 44100  # on défini de la fréquence d'échantillonnage à 44 100 Hz
duree = 5  # et la durée de l'enregistrement à 5 secondes
monenregistrement = sd.rec(int(duree*fe), samplerate=fe, channels=2)
# Enregistrement d'un son pendant la durée spécifiée
sd.wait()  # Attend la fin de l'enregistrement
sd.play(monenregistrement, fe)
# on lit de l'enregistrement à la fe définie plus haut
```

Pour la suite on considèrera avoir importé les librairies et modules si dessous ainsi que le code Ma312_BE_lib contenant les fonctions dont nous aurons besoin :

```
import sounddevice as sd
import time
from scipy.io import wavfile
import numpy as np
from matplotlib.pyplot import *
from numpy.fft import fft
import math
import soundfile as sf
from scipy import signal
import os


import Ma312_BE_lib as lib
```

5.1 Exemple d'un signal par addition de plusieurs fréquences

On peut générer un signal en additionnant plusieurs sinusoïdes de fréquences différentes. Ici par exemple, nous générons un signal par l'addition de trois sinusoïdes aux fréquences $523Hz$, $659Hz$ et $392Hz$.

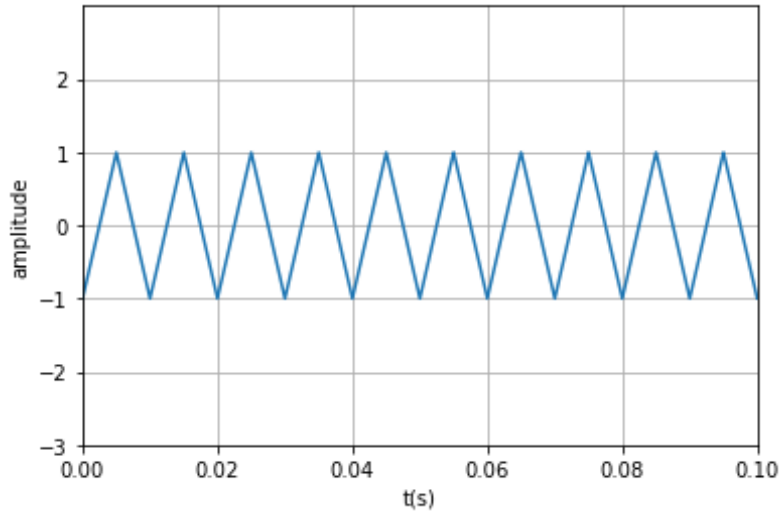
```
amplitude = 2  #  approx volume du son.
freq1 = 523  # Fréquence de la première sinusoïde (523 Hz = DO5)
freq2 = 659  # Fréquence de la deuxième sinusoïde (659 Hz = MI5)
freq3 = 392  # Fréquence de la troisième sinusoïde (392 Hz = SOL4)
duree = 2  # Durée du signal généré toujours en secondes
fe = 44100

t = np.linspace(0, duree, fe*duree)  # Génération d'une grille de temps.
signal = (amplitude * np.sin(2 * np.pi * t * freq1)
          + amplitude * np.sin(2 * np.pi * t * freq2)
          + amplitude * np.sin(2 * np.pi * t * freq3))
sd.play(signal)  # et on le joue
te = 1.0 / fe  # Calcul de la période d'échantillonnage
n = signal.size  # Réécupère la taille du signal
t2 = np.arange(n) * te  # Génération des points temporels
figure()  # on crée une nouvelle figure pour dessiner un graphique
plot(t2, signal)  # tracé du signal dans le domaine temporel
xlabel("t(s)")  # donne un nom à l'axe des abscisses (temps en seconde)
ylabel("amplitude")  # et à l'axe des ordonnées (amplitude du signal)
axis([0, 0.05, signal.min()-2, signal.max()+2])  # on définie les limites des axes
grid()  # ajoute une grille en fond du graphique
print(signal.max())  # affiche dans la console la valeur de l'amplitude maximale du signal
```

Remarque : Vous pouvez essayer de générer un signal en additionnant 2 sinusoïdes avec un déphasage de π et vous obtiendrez alors un signal nul.

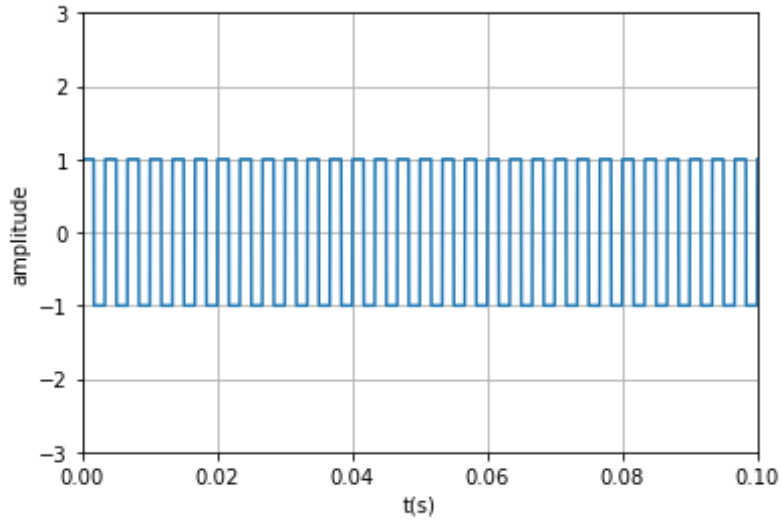
Il peut être interessant d'observer les différences obtenues pour des signaux triangulaires ou carrés, que l'on trace ici en temporel.
Pour un signal triangulaire :

```
amplitude = 2
duree = 2
fe = 44100
t = np.linspace(0, duree, fe*duree)
triangle = signal.sawtooth(2*np.pi*100*t, 0.5)
sd.play(triangle)
te = 1.0 / fe
n = triangle.size
t2 = np.arange(n) * te
figure()
plot(t2, triangle)
xlabel("t(s)")
ylabel("amplitude")
axis([0, 0.1, triangle.min()-2, triangle.max()+2])
grid()
```

Pour un signal carré :




```
amplitude = 2
duree = 2
fe = 44100
t = np.linspace(0, duree, fe*duree)
square = signal.square(2*np.pi*300*t, 0.5)  # génération d'un signal carré
sd.play(square)
te = 1.0 / fe
n = square.size
t2 = np.arange(n) * te
figure()
plot(t2, square)
xlabel("t(s)")
ylabel("amplitude")
axis([0, 0.1, square.min()-2, square.max()+2])
grid()
```



5.2 "Ah vous dirai-je Maman"

On peut utiliser le code précédent en le prolongeant simplement pour qu'il joue une succession de notes. Grâce à lui on peut écouter le "Ah vous dirai-je Maman" :

```
tempo=1/2
amplitude=1
echantillonage=44100
temps=tempo/4
pause=np.linspace(0,temps,int(echantillonage*temps))
freq=262 # do
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ([ note , pause , note ])
sd.play(signal)
freq=392 # sol
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ([ signal , pause , note , pause , note ] )
sd.play(signal)
freq=440 # la
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , pause , note , pause , note ] )
sd.play(signal)
freq=392 # sol
temps=tempo
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , pause , note ] )
sd.play(signal)
freq=349 # fa
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , 1.5*pause , note , pause , note ] )
sd.play(signal)
freq=330 # mi
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , pause , note , pause , note ] )
sd.play(signal)
freq=294 # re
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , pause , note , pause , note ] )
sd.play(signal)
```

```
freq=262 # do
temps=tempo
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , pause , note ] )
sd.play(signal)
freq=392 # sol
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , 1.5*pause , note , pause , note ] )
sd.play(signal)
freq=349 # fa
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , pause , note , pause , note ] )
sd.play(signal)
freq=330 # mi
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , pause , note , pause , note , pause , note ] )
sd.play(signal)
freq=294 # re
temps=tempo/2
t=np.linspace(0,temps,int(echantillonage*temps))
note=amplitude* np.sin(2* np.pi *t* freq )
signal=np.block ( [ signal , pause , note ] )
sd.play(signal)
```

Après exécution on a le plaisir d'écouter le morceau ! Cependant, on remarque que le code ci-dessus n'est pas optimisé. On cherche maintenant un moyen de faciliter l'écriture de ce programme en conservant le résultat précédent.

Pour ce faire, il est possible d'aller plus vite et de réduire drastiquement le code en utilisant une boucle et 2 listes contenant les notes à jouer et le rythme auquel les jouer :

```python
#définition de chaque fréquance pour toutes les notes d'une gamme de do

do = 261.63
re = 293.66
mi = 329.63
fa = 349.23
sol = 392
la = 440
si = 493.78

# Paramètres de la mélodie
tempo = 1/2
amplitude = 1
echantillonage = 44100
pause_duree = tempo / 4  # Durée d'une pause entre les notes
pause = np.zeros(int(echantillonage * pause_duree))  # Signal de silence (pour les pauses)

# Fonction pour générer une note
def generer_note(freq, duree, echantillonage):
    t = np.linspace(0, duree, int(echantillonage * duree), endpoint=False)
    return amplitude * np.sin(2 * np.pi * freq * t)

# Séquence des notes et leurs durées (en unités de tempo défini plus haut)
partition = [do, do, sol, sol, la, la, sol, fa, fa, mi, mi, re, re, do]
durees = [2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]

# Création du signal complet
signal_complet = np.array([])

#boucle qui se répète autant de fois qu'il y a de valeur dans la liste "partition"
for i in range(len(partition)):
    duree_note = tempo / durees[i]
    note = generer_note(partition[i], duree_note, echantillonage)
    signal_complet = np.concatenate((signal_complet, note, pause))

# Lecture de la mélodie
sd.play(signal_complet, echantillonage)
sd.wait()
sd.play(signal)
```

5.3 Importation d'un fichier et tracé en fréquenciel

Comme sur le premier exemple, on récupère le chemin du fichier "guitare1.wav" et on l'importe.

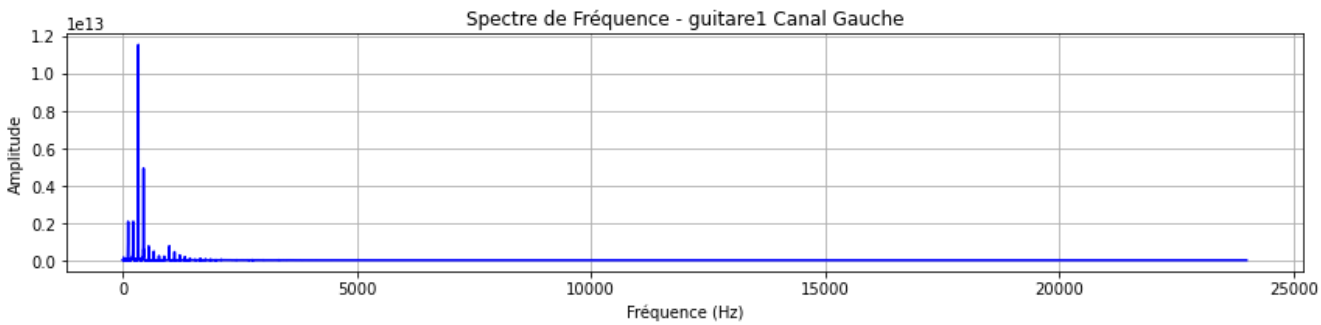
```python
file_path = os.path.join('Data', 'guitare1.wav')
fe, data = wavfile.read(file_path)
print(f"Taille de l'array de {nom_fichier} :", data.shape)
```

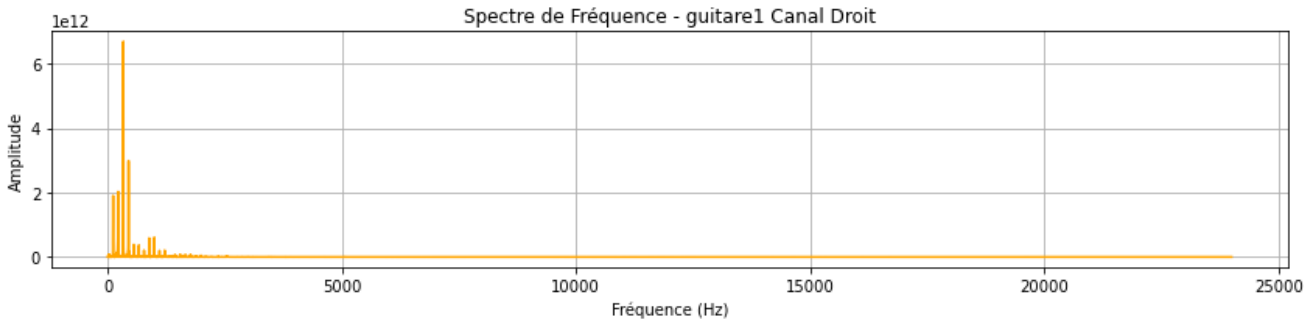
L'exécution de ce programme permet d'afficher dans la console :
(le nombre d'échantillons, le nombre de canaux)
Ici 383407 échantillons pour 2 canaux (= stéréo).
On extrait le nom du fichier sans l'extension pour l'ajouter automatiquement dans le titre du graphe plus tard.

```
nom_fichier = os.path.splitext(os.path.basename(file_path))[0]
```

On veut maintenant vérifier que le fichier est bien stéréo :

```
if len(data.shape) > 1:
    print("Le fichier est stréreo")
    canal_gauche = data[:, 0]  # Premier canal (gauche)
    canal_droit = data[:, 1]    # Deuxième canal (droit)

    # on applique la FFT
    fft_gauche = np.fft.rfft(canal_gauche)
    fft_droit = np.fft.rfft(canal_droit)

    # puis on calcule les fréquences correspondantes
    freqs = np.fft.rfftfreq(len(canal_gauche), d=1./fe)
    figure(figsize=(12, 6)) # Tracer les spectres de fréquence

    # Spectre de fréquence pour le canal gauche
    subplot(2, 1, 1)
    plot(freqs, np.abs(fft_gauche), color='blue')
    title(f'Spectre de Fréquence - {nom_fichier} Canal Gauche')
    xlabel('Fréquence (Hz)')
    ylabel('Amplitude')
    tight_layout()  # Ajuste les graphes pour ne pas se chevaucher
    grid()

    # Spectre de fréquence pour le canal droit
    subplot(2, 1, 2)
    plot(freqs, np.abs(fft_droit), color='orange')
    title(f'Spectre de Fréquence - {nom_fichier} Canal Droit')
    xlabel('Fréquence (Hz)')
    ylabel('Amplitude')
    grid()
    tight_layout()
    show()
else:
    print("Le fichier audio est mono")
    # on applique donc la FFT au seul canal
    fft_mono = np.fft.rfft(data)
    freqs = np.fft.rfftfreq(len(data), d=1./fe) # Calcul des fréquences

    # Tracer le spectre de fréquence
    figure(figsize=(12, 6))
    plot(freqs, np.abs(fft_mono), color='green')
    title(f'Spectre de Fréquence - {nom_fichier}')
    xlabel('Fréquence (Hz)')
    ylabel('Amplitude')
    grid()
    tight_layout()
    show()
```





5.4 Et avec des puissances de 2

On peut utiliser le même code mais en appliquant la transformée de Fourier rapide sur un échantillon.
Le plus grand contenu dans la taille de notre array.
La transformée de Fourier est plus rapide si la taille des échantillons sont des puissances de 2.

```
file_path = os.path.join('Data', 'guitare1.wav')
fe, data = wavfile.read(file_path)

nom_fichier = os.path.splitext(os.path.basename(file_path))[0]
print(f"Taille de l'array de {nom_fichier} :", data.shape)



canal_gauche = data[:, 0]
canal_droit = data[:, 1]
```

On veut trouver la plus grande puissance de 2 dans la taille de notre array. On utilise la librairie Numpy.

```
n = len(canal_gauche)
puissance_de_2 = 1 << (n - 1).bit_length()
```

On vérifie que la puissance ne dépasse pas la taille de l'array sinon on prend celle d'avant.

```
if puissance_de_2 > n:
   puissance_de_2 //= 2

   print("La plus grande puissance de 2 est : ", puissance_de_2)
```

On va extraire les données de chaque canal jusqu'à la valeur de la plus grande puissance de 2 trouvée

```
canal_gauche_2 = canal_gauche[:puissance_de_2]
canal_droit_2 = canal_droit[:puissance_de_2]

 # On applique la RFFT
fft_gauche = np.fft.rfft(canal_gauche_2)
fft_droit = np.fft.rfft(canal_droit_2)

freqs = np.fft.rfftfreq(len(canal_gauche_2), d=1./fe)

figure(figsize=(12, 6))
```

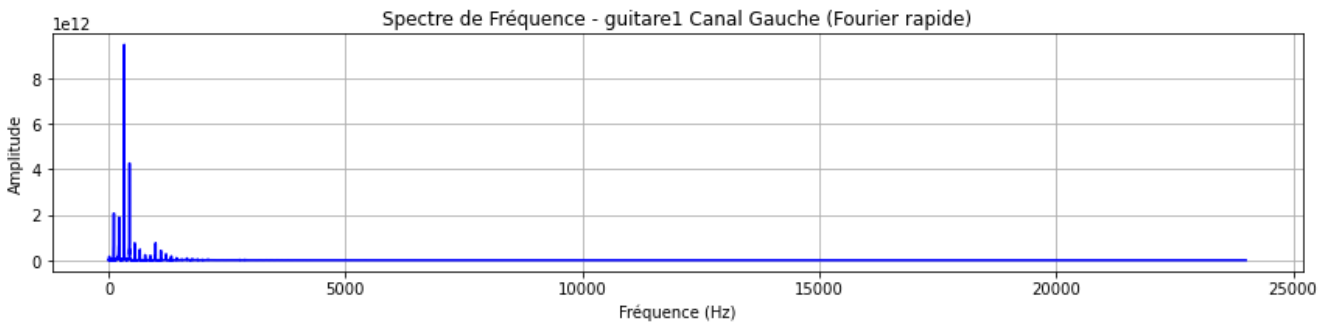
Tracé des spectres :

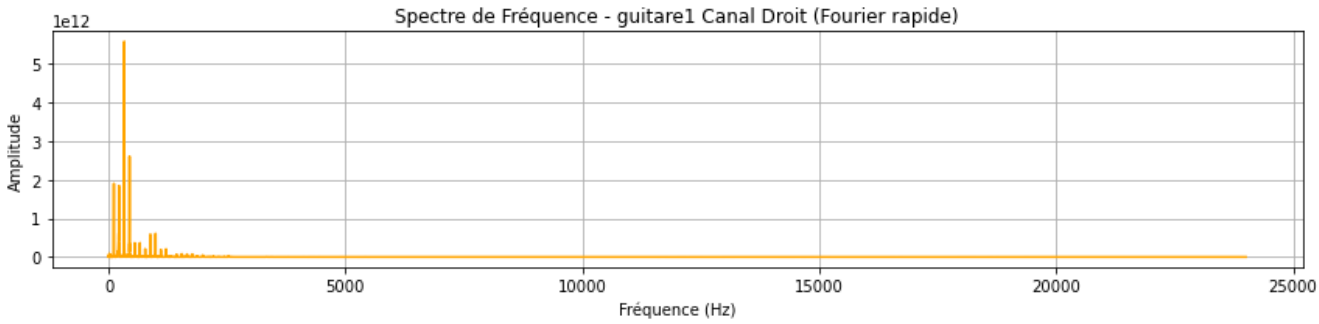
```
 # Tracé pour spectre de fréquence pour le canal gauche
subplot(2, 1, 1)
plot(freqs, np.abs(fft_gauche), color='blue')
title(f'Spectre de Fréquence - {nom_fichier} Canal Gauche (plus grande puissance de 2)')
xlabel('Fréquence (Hz)')
ylabel('Amplitude')
grid()

 # Tracé pour spectre de fréquence pour le canal droit
subplot(2, 1, 2)
plot(freqs, np.abs(fft_droit), color='orange')
title(f'Spectre de Fréquence - {nom_fichier} Canal Droit (plus grande puissance de 2)')
xlabel('Fréquence (Hz)')
ylabel('Amplitude')
grid()

tight_layout()

show()
```

5.5 Autre méthode : l'auto-corrélation

L'objectif est de déterminer la note jouée à l'aide de cette nouvelle méthode.

```
file_path = os.path.join('Data', 'guitare1.wav')
fe, data = wavfile.read(file_path)

nom_fichier = os.path.splitext(os.path.basename(file_path))[0]
```






```python
# On récupère les notes MIDI et leurs noms depuis le .txt grace à cette fonction
def charger_notes(fichier):
    notes = {}
    # On ouvre le fichier spécifié en mode lecture seule
    with open(fichier, 'r') as file:
        # Puis on parcourt chaque ligne du fichier
        for line in file:
            index, name = line.split(": ")  # On sépare la ligne nom MIDI et nom francais
            # Convertit l'index en entier et ajoute l'entrée au dictionnaire
            notes[int(index)] = name.strip().strip('"')
    return notes

# on charge le son que l'on veut étudier
file_path = os.path.join('Data', 'guitare1.wav')
fe, data = wavfile.read(file_path)

nom_fichier = os.path.splitext(os.path.basename(file_path))[0]

# Vérification du format de l'audio
if len(data.shape) == 1:  # Si mono
    canal_gauche = data
else:  # Si stéréo
    canal_gauche = data[:, 0]  # On utilise le canal gauche ici

# Normalisation
canal_gauche = canal_gauche / np.max(np.abs(canal_gauche))

# On calcule l'auto-corrélation
auto_corr = np.correlate(canal_gauche, canal_gauche, mode='full')

# Puis on la dérive
derivative = np.diff(auto_corr)

# Trouver les pics dans la dérivée
peaks = np.where((derivative[:-1] > 0) & (derivative[1:] <= 0))[0] + 1
```

```python
# Calculer les intervalles entre les grands pics
threshold = np.max(auto_corr) * 0.5  # Seuil pour considérer un pic "grand"
grand_pics = peaks[auto_corr[peaks] > threshold]


# On récupère toutes les notes MIDI et leur nom en francais
chemin_MIDI = os.path.join('Data', 'notes_MIDI.txt')
notes_midi = charger_notes(chemin_MIDI)


if len(grand_pics) >= 2:
    interval = grand_pics[1] - grand_pics[0]
    frequency = fe / interval  # Fréquence de la note
    note_nombre = 69 + 12 * np.log2(fe / 440.0)  # Note MIDI
    note_nom = int(round(note_nombre))  # Arrondi pour avoir la note MIDI la plus proche

# Puis on retrouve le nom de la note en français grâce au fichier qui les combine toutes
    if note_nom in notes_midi:
        print(f"La note est donc : {notes_midi[note_nom]}")
    else:
        print("Note non reconnue, il faut vous accorder.")
else:
    print("On ne peut pas déterminer la note, pas assez de pics détectés.")
```

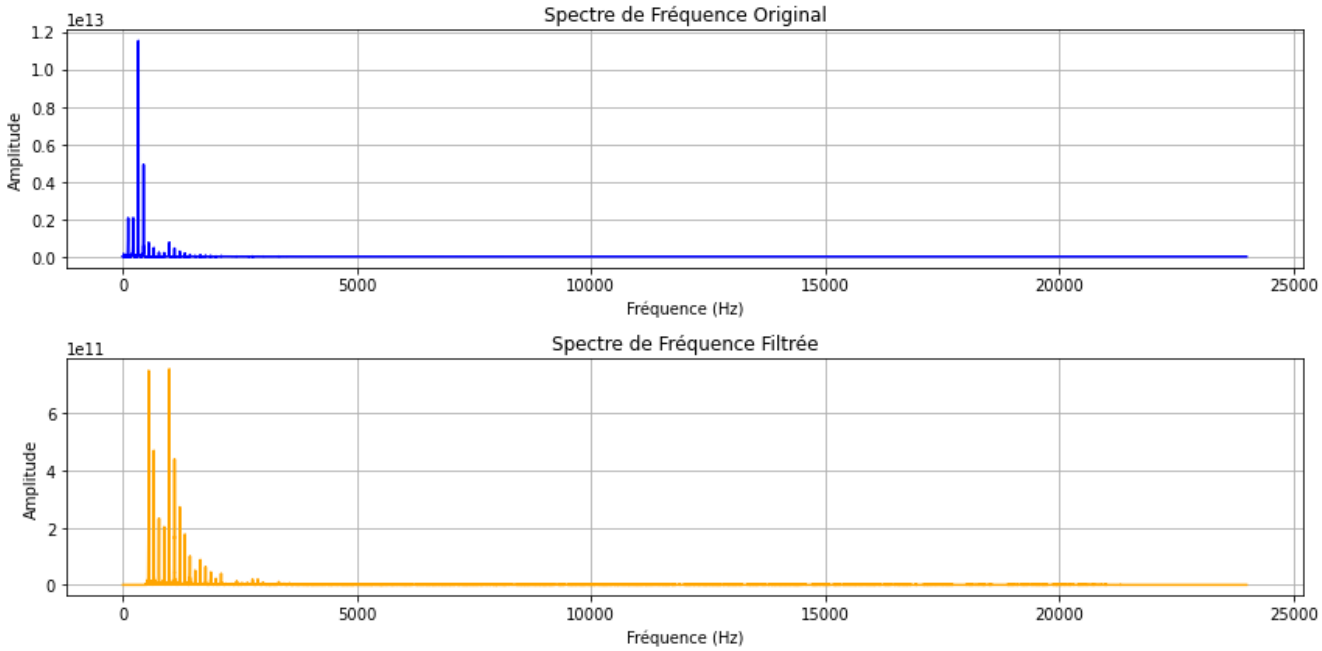
On peut aussi tracer graphiquement l'auto-corrélation et lire nous même le résultat en vérifiant la fréquence

5.6 Reconstruire un son après avoir enlevé certaines fréquences

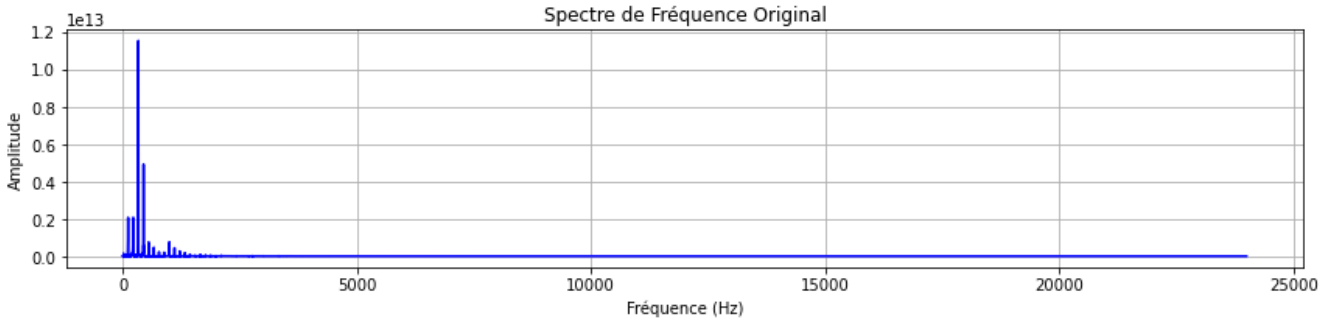
```python
file = os.path.join('Data', 'guitare1.wav')
fe, data = wavfile.read(file)

# On utilise toujours le canal de gauche ici
canal_gauche = data[:, 0]

# on recalcule la FFT et les fréquences
fft_gauche = np.fft.rfft(canal_gauche)
freqs = np.fft.rfftfreq(len(canal_gauche), d=1./fe)

# On ajoute une copie pour appliquer le filtre sans modifier l'original
fft_filtrée = np.copy(fft_gauche)

# on supprime certaines fréquences ici de 0 à 500
frequences_a_supprimer = (freqs >= 0) & (freqs <= 500)
fft_filtrée[frequences_a_supprimer] = 0

# on reconsturit le signal avec la transformée de Fourier inverse
canal_reconstruit = np.fft.irfft(fft_filtrée)

canal_reconstruit /= np.max(np.abs(canal_reconstruit))  # Normaliser entre -1 et 1

sd.play(canal_reconstruit, fe)
sd.wait()
```

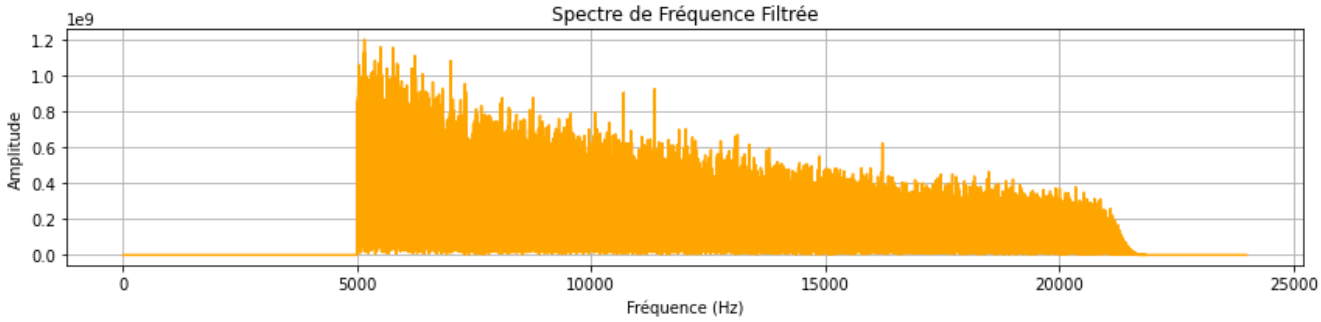
On trace alors le spectre d'origine et le spectre filtré puis reconstruit pour les comparer.

```
figure(figsize=(12, 6))

# Spectre original
subplot(2, 1, 1)
plot(freqs, np.abs(fft_gauche), color='blue')
title('Spectre de Fréquence Original')
xlabel('Fréquence (Hz)')
ylabel('Amplitude')
grid()

# Spectre filtré
subplot(2, 1, 2)
plot(freqs, np.abs(fft_filtrée), color='orange')
title('Spectre de Fréquence Filtrée')
xlabel('Fréquence (Hz)')
ylabel('Amplitude')
grid()

tight_layout()
show()
```



Nous pouvons remarquer que de nouveaux pics sont apparus sur le spectre filtré et reconstruit. Il n'y a rien de 0 à $500Hz$ mais une nouvelle note sur les fréquences un peu plus hautes.





Nous pouvons ensuite augmenté la puissance du filtre de facon à bloquer toute la note, nous poussons son efficacité. Ici, il n'y a plus de signal avant 5 000Hz, apparaît alors du bruit à une amplitude bien plus faible (voir le changement d'échelle) mais qui produit bien un son de par la reconstruction.

5.7 Exemple de décalage en fréquence

En décalant en fréquence la transformée de Fourier et en utilisant la transformée de Fourier inverse, on cherche à jouer un Do (131 Hz) avec cette guitare.

```
def jouer_note_decale(data, fe, freq_note):
    fft_data = np.fft.rfft(data) # on calcule la FFT
    freqs = np.fft.rfftfreq(len(data), d=1./fe) #on trouve la fréquence associée

    note_freq = freq_note #on décale la FFT
    shift_amount = note_freq / fe  # on le normalise

    fft_decale = np.roll(fft_data, int(len(fft_data) * shift_amount))

    signal_reconstruit = np.fft.irfft(fft_decale) # puis on le reconstitue le signal avec la transformé

    signal_reconstruit /= np.max(np.abs(signal_reconstruit))  # on le normalise

    sd.play(signal_reconstruit, fe) #puis on le joue
    sd.wait()
```

```
fichier_musique = 'Data/guitare1.wav'
fe, data = wavfile.read(fichier_musique)

# Supposons que l'on utilise le canal gauche
canal_gauche = data[:, 0]

#on utilise la fonction de décalage pour jouer avec le son de guitare un DO (523 Hz)
lib.jouer_note_decale(canal_gauche, fe, 523)

fichier_musique = os.path.join('Data', 'piano.wav')
fe, data = wavfile.read(fichier_musique)


signal_effet = lib.delay(fe, data)

sd.play(signal_effet, data)
sd.wait()
```

5.8 Générer un effet sonore : le delay

On souhaite tester l'effet delay. Pour ce faire, nous allons jouer avec plusieurs paramètres.
L'objectif va être de générer un écho qui aura pour conséquence de nous faire entendre un son plus fort.

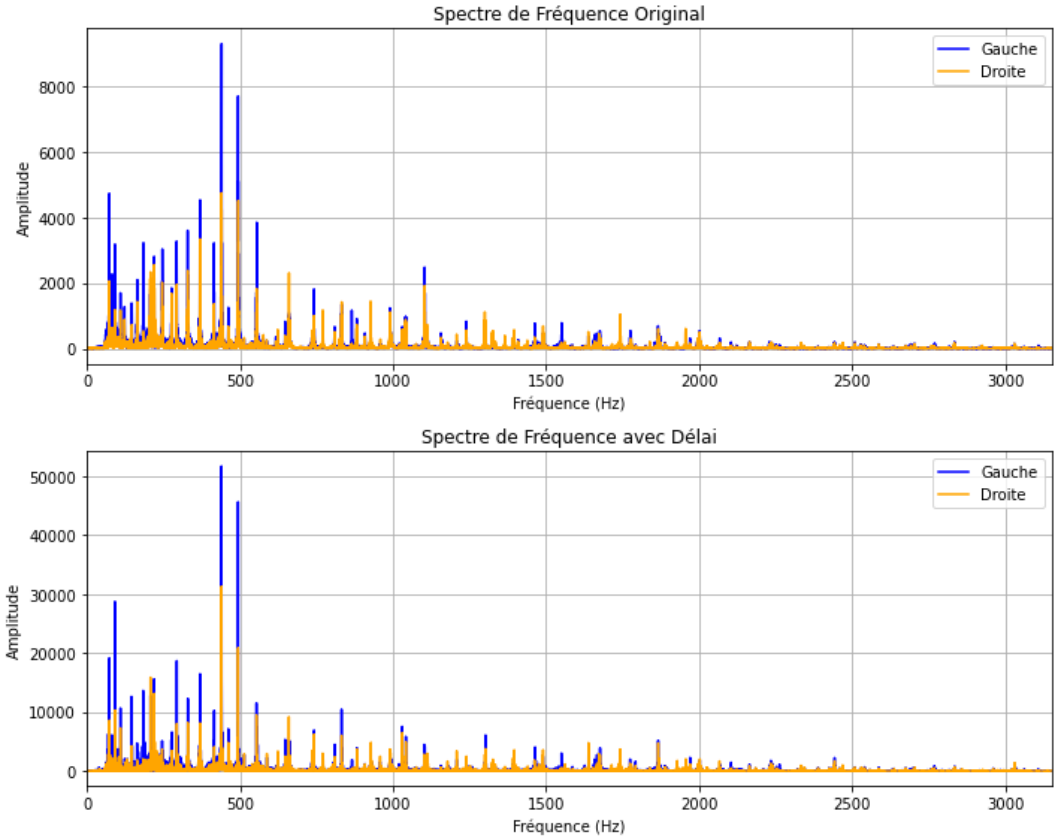
On modifie donc le son piano.wav avec l'effet delay :

```
file = "Data/piano.wav"

fe, data = sf.read(file)

# on applique l'effet de délai sur chaque coté du stéréo,

#ATTENTION AUX OREILLES MODIFICATEURS A FOND A DROITE

delayed_signal_l = delay(fe[:, 0], data,0.5,1,1)
delayed_signal_r = delay(fe[:, 1], data,0.5,5,5)
delayed_signal = np.column_stack((delayed_signal_l, delayed_signal_r))

# puis on calcule les spectres
freqs = np.fft.rfftfreq(len(fe), d=1/data)
fft_original_l = np.abs(np.fft.rfft(fe[:, 0]))
fft_original_r = np.abs(np.fft.rfft(fe[:, 1]))
fft_delayed_l = np.abs(np.fft.rfft(delayed_signal[:, 0]))
fft_delayed_r = np.abs(np.fft.rfft(delayed_signal[:, 1]))

# pour jouer le fichier audio avec effet de délai
sd.play(delayed_signal, data)
sd.wait()

figure(figsize=(10, 8))
```

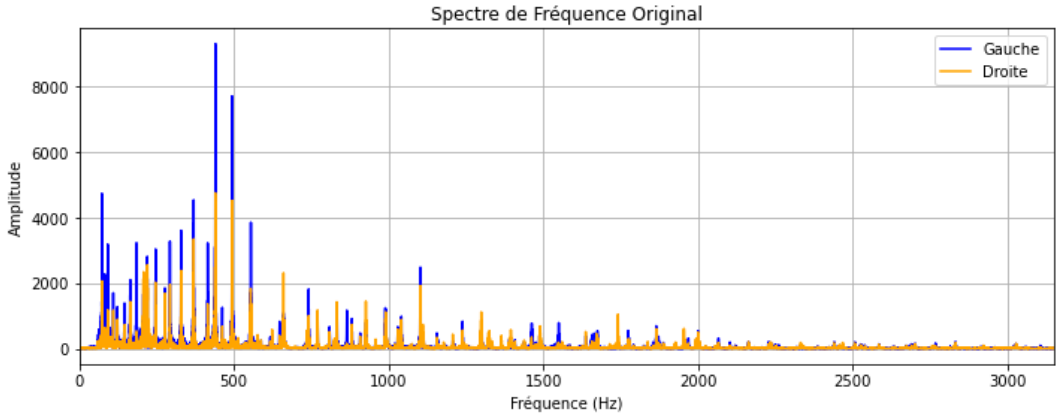
Il est interessant d'observer les spectres du signal original et du signal modifié par la fonction delay :

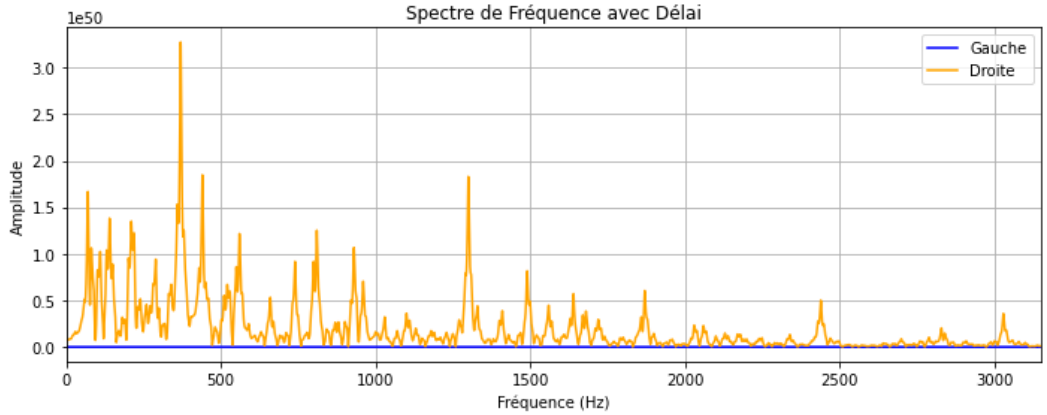
```
# Spectre original
subplot(2, 1, 1)
plot(freqs, fft_original_l, color='blue', label='Gauche')
plot(freqs, fft_original_r, color='orange', label='Droite')
title('Spectre de Fréquence Original')
xlabel('Fréquence (Hz)')
ylabel('Amplitude')
xlim(0, data/14)
grid()
legend()

# Spectre avec délai
subplot(2, 1, 2)
plot(freqs, fft_delayed_l, color='blue', label='Gauche')
plot(freqs, fft_delayed_r, color='orange', label='Droite')
title('Spectre de Fréquence avec Délai')
xlabel('Fréquence (Hz)')
ylabel('Amplitude')
xlim(0, data/14)
grid()
legend()
tight_layout()
show()
```

En prenant un Delay modéré de (0.5, 1, 1), on obtient les graphes suivants :



On obtient une amplitude maximale d'environ 50 000 pour le spectre avec Delay contre une amplitude maximale de 9 000 pour le spectre de fréquence original.
Avec une valeur de Delay exagérée (0.1, 5, 5), on obtient les graphes suivants :





Déjà nous perdons nos oreilles mais aussi, ces deux derniers graphes nous permettent de voir que la courbe correspondant au coté droit monte à des $1e^50$, bien au dessus de celle de gauche qui paraît plate à côté.
Ces deux tests nous permettent de nous rendre plus facilement compte des conséquences de la fonction delay sur notre son et d'à quel point il et amplifié par ces répétitions.

5.9 Convolution de réponse impulsionnelle

On veut réaliser une convolution en utilisant la méthode du zéro-padding pour compléter les vecteurs de taille M et N à la taille $M + N - 1$.
Nous pouvons effectuer des convolutions avec deux vecteurs qui n'ont pas la même taille et cette méthode évite les problèmes de valeurs aux bords.
La taille de la sortie de la convolution de deux signaux de tailles M et N est $M + N - 1$ car chaque échantillon d'un coté peut interagir avec chaque échantillon de l'autre. Nous avons besoin de $M + N - 1$ indices pour tout couvrir.

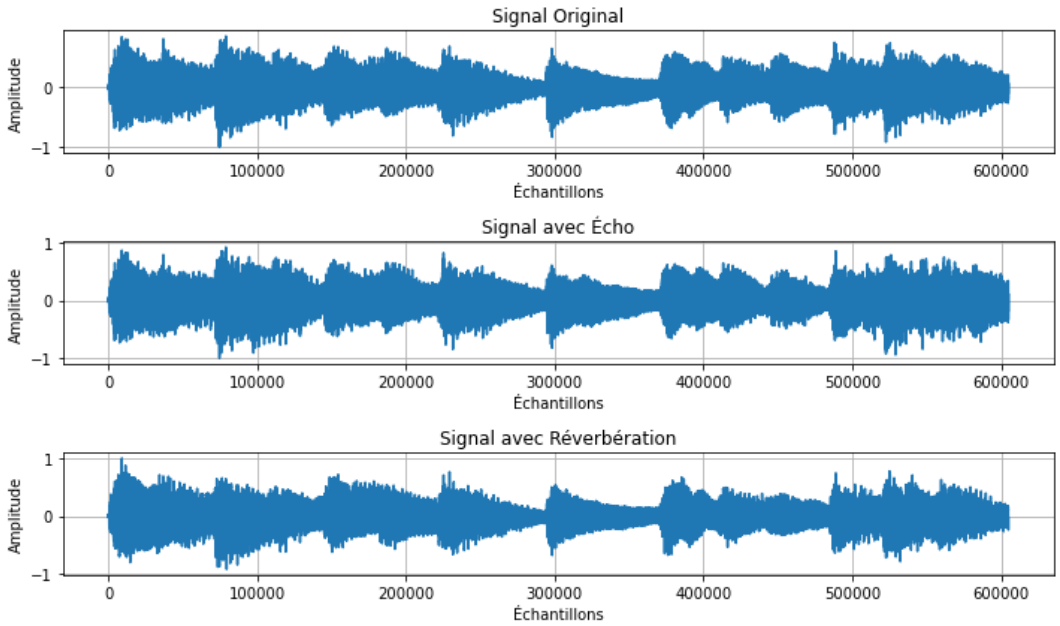
```python
file = "Data/piano.wav"
fe, data = sf.read(file)

fe = np.ravel(fe)  # Aplatir le signal pour gagner en mémoire (le son ne voulait
                    #pas être joué autrement)

# Appliquer les réponses impulsionnelles
echo_ir = impulse_response_echo(delay=22050)  # Écho de 0.5 seconde
reverb_ir = impulse_response_reverb(length=44100, decay=0.9) # Réverbération de 0.1 s

# Convoler le signal avec l'écho et la réverbération
signal_echo = conv_fft(fe, echo_ir)
signal_reverb = conv_fft(fe, reverb_ir)

signal_echo /= np.max(np.abs(signal_echo))  # Normalisation de l'écho
signal_reverb /= np.max(np.abs(signal_reverb))  # Normalisation de la réverbération
```

On joue les résultats en ajoutant un facteur 2 car ils ont été ralenti dans l'opération :

```python
factor = 2.0
print("Jouer le signal d'origine...")
sd.play(fe, data*factor)
sd.wait()

print("Jouer le signal avec écho...")
sd.play(signal_echo, data*factor)
sd.wait()

print("Jouer le signal avec réverbération...")
sd.play(signal_reverb, data*factor)
sd.wait()
```

Puis enfin, comme à chaque fois on finit par tracer les signaux obtenus avec la même méthode que pour les autres spectres :

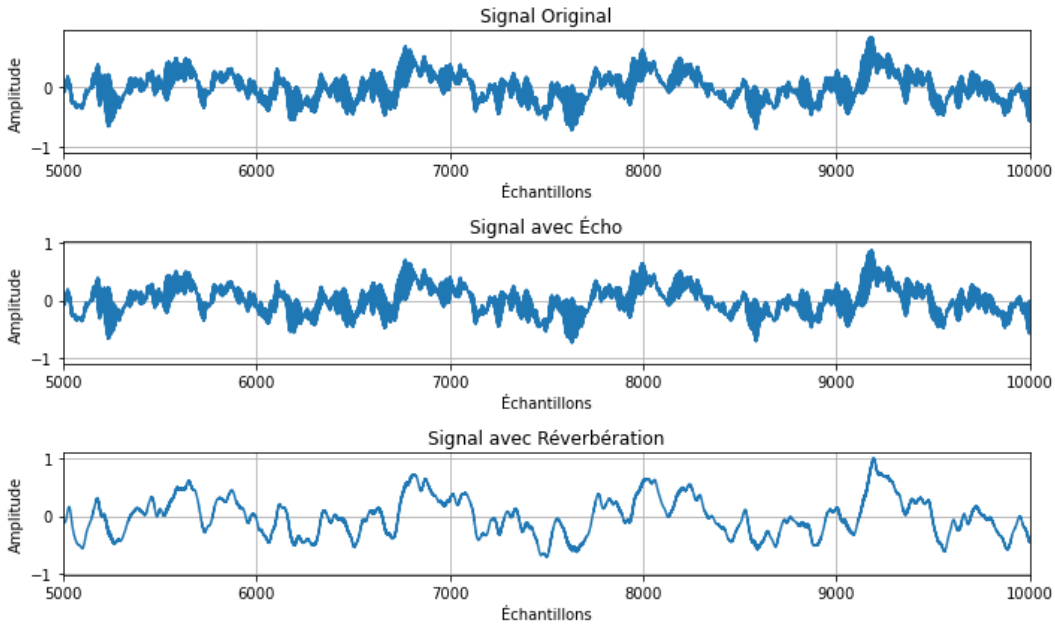
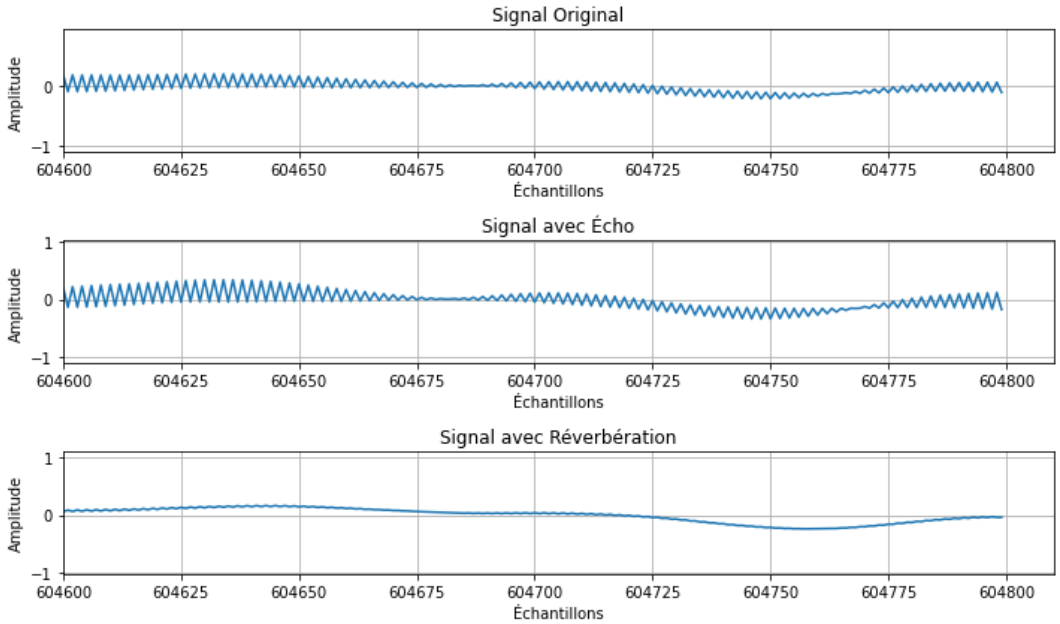
```
figure(figsize=(10, 6))

# Signal original
subplot(3, 1, 1)
plot(fe)
title('Signal Original')
xlabel('Échantillons')
ylabel('Amplitude')
grid()

# Signal avec écho
subplot(3, 1, 2)
plot(signal_echo)
title('Signal avec Écho')
xlabel('Échantillons')
ylabel('Amplitude')
grid()

# Signal avec réverbération
subplot(3, 1, 3)
plot(signal_reverb)
title('Signal avec Réverbération')
xlabel('Échantillons')
ylabel('Amplitude')
grid()

tight_layout()
show()
```



En regardant le spectre entier du signal, on ne remarque pas de différence flagrante, ce ne sont pas exatement les même pour autant. Nous pouvons donc zoomer un peu en ajoutant pour chaque signal :

```
xlim(5000, 10000) # avec les valeurs que l'on souhaite observer
```

Dans ce cas, nous obtenons ceci :



Les deux premiers ont un trait "plus épais" mais nous ne voyons pas encore pourquoi. Si nous limitons x sur l'intervalle encore plus fin $(604600, 604800)$ par exemple, nous pouvons déjà remarquer que les trois signaux se terminent en même temps mais surtout que la réverberation change énromément l'allure de la courbe avec une amplitude d'ondulation bien plus faible, contrairement à l'écho qui l'augmente légèrement :